Bomb Party — PRD

Overview

A phone-based party game for groups playing together in the same room. A category comes up, players take turns naming something that fits before a randomized fuse runs out. Miss it, take a letter toward "BOMB"; spell all four and you're eliminated. Last player standing wins. Phones are the controller — no app to install, just a link.

Goal

Something playable at an actual gathering this weekend, with zero install friction and no moment where the tech gets in the way of the game. Done means a group of friends can open a link, play a full game start to finish, and want to run it again — not that it looks like a shipped product.

Who it's for

4-10 people, physically together, phones already in hand. Not built for remote play in v1 (see Out of scope).

Core gameplay loop

1. A category is announced ("fruits," "80s movies") and a fuse length is drawn at random — long enough to feel fair, short enough to stay tense (10-40 seconds as a starting range).
2. The current player says something that fits the category out loud, then taps Pass before the fuse hits zero.
3. Clean pass: the bomb moves to the next player, the category stays the same, and a brand new random fuse is drawn — so nobody can count along and predict it.
4. Miss (fuse hits zero first): that player takes one letter toward B-O-M-B, and a new category is drawn for the next player.
5. Four letters and a player is eliminated from the rotation.
6. Last player remaining wins.

Answers are judged by the room, not the software — everyone's already there in person, talking. Same goes for duplicate answers within a category: the group polices it, the app doesn't need to.

Functional requirements (v1)

- Create a room and get a shareable code/link
- Join from a phone browser with just a nickname — no account, no install
- Turn order management across all connected players
- Category selection from a static list, persisting across turns until a miss
- Randomized fuse duration per turn, synced across all clients from one server-side deadline
- Pass action that advances the turn and re-rolls the fuse
- Letter/strike tracking per player, with elimination at four letters
- Game-over state once one player remains
- A disconnecting and rejoining phone doesn't break the room

Non-functional requirements

- Comfortably handles up to ~10 concurrent players in a room
- Fuse should feel synced across phones — drift under a few hundred milliseconds is fine, since nothing here is judging who answered first, just when the bomb goes off
- No data needs to survive a server restart — in-memory state is enough
- Runs entirely on a local network with no internet dependency, as the default mode

Out of scope (v1)

- Word or dictionary validation — deliberately left to the room
- Enforced duplicate-answer detection
- Accounts, persistence, or history across sessions
- A native mobile app
- Remote play over the open internet (reachable later via ngrok, not a v1 goal)
- A separate shared-screen host view — a good add-on, not required to play

Data model

Room: `room_code`, `players[]`, `current_player_index`, `current_category`, `deadline_timestamp`, `used_categories`

Player: `id`, `nickname`, `letters` (subset of B/O/M/B), `eliminated`, `connected`

Technical approach

- Backend: FastAPI with native WebSocket support — no extra framework needed at this scale. `python-socketio` as a fallback if hand-rolled reconnect logic gets annoying.

- State: one in-memory dict of `room_code` to Room. No database.
- Frontend: plain HTML/CSS/JS, no build step — a join screen, an active-turn screen, a spectating screen, and a game-over screen cover the whole game.
- Hosting: run locally on the host's laptop, bound to `0.0.0.0`, joined via local IP or a QR code over shared wifi. ngrok as a fallback if the network blocks device-to-device traffic or players aren't on the same wifi.

Build phases

1. Core loop, single device — get the room/turn/fuse state machine right with browser tabs standing in for players.
2. Multi-device sync — same logic, real phones on the same wifi, confirm the fuse actually feels synced.
3. Polish — QR-code join flow, sound and visual drama on the fuse and on elimination, optional host/shared-screen view.

Open questions

- Exact fuse min/max, and whether it should shrink as players are eliminated
- Whether the host/shared-screen view makes it into v1 or gets cut for time
- Category list source — hand-write one, or pull from an existing public word-game dataset

Success criteria

- A full game plays start to finish with at least 4 people, no crash or stuck state
- The fuse feels synced enough that nobody notices phones are out of sync
- Someone at the party asks to play again